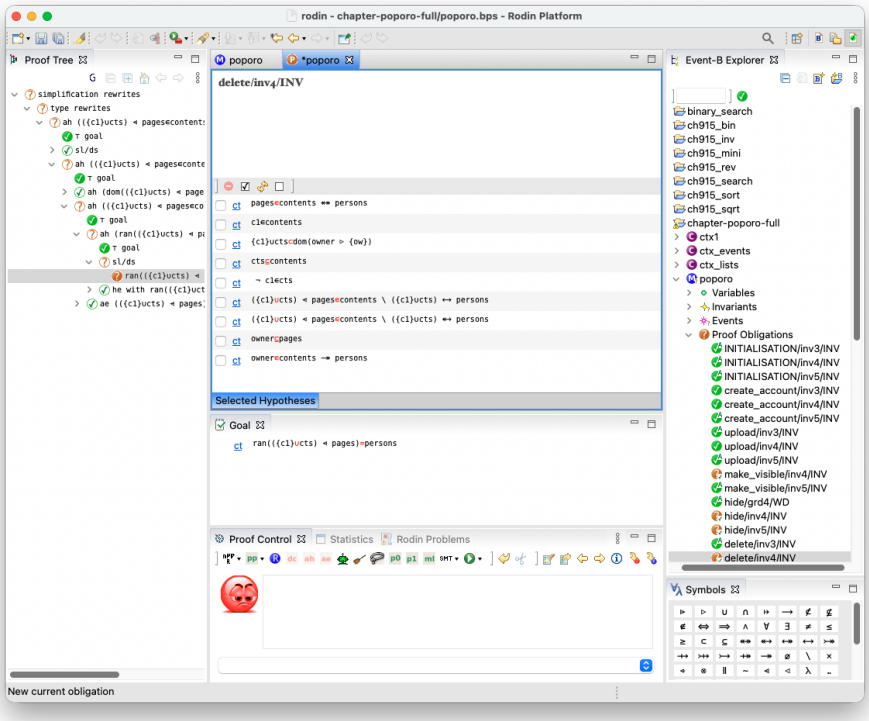

Fig. 3.2: Rodin PO editor

3.4 EventB2Java

The EventB2Java tool [2] is a Rodin [1] plug-in. It translates Event-B models into Java code. A key feature of EventB2Java is the generation of JML formal specifications along with the Java code. This feature enables users to write custom code that replaces the code generated by EventB2Java, and then use existing JML tools (https://www.cs.-ucf.edu/ leavens/JML/) to verify that the custom code is correct. JML is a behavioural specification language for Java. JML annotations are generated within the code as Java comments so that Java tools will interpret the annotations as code documentation and JML tools as code specifications. There exist three main techniques to execute Event-B models [4]: (*a*) *Translation* is a technique to generate fully deterministic models with data implementations that are native to the target language (e.g., Java, C, or Rust). This technique can only be used towards the end of an Event-B refinement chain. (*b*) *Animation* is an implementation of the operational semantics of Event-B; it can be applied to Event-B models with a moderate level of determinism. (*c*) *Simulation* is a technique that combines Translation with Animation. Non-determinism is addressed by allowing users to provide implementations of non-executable expressions. Although Translation

is typically applicable only to fully refined and deterministic models, EventB2Java extends its applicability earlier in the refinement chain by using ProB's constraint solver to instantiate non-deterministic and underspecified expressions with concrete values.
EventB2Java uses the Rodin API to collect all components of the machine to be translated, *e.g.*, carrier sets, constants, axioms, variables, invariants and events, as well as all the necessary information from the refined machines. All this information is stored in the Rodin database. EventB2Java parses expressions and statements as abstract syntax trees using the AST library provided by Rodin. The Rodin API also provides a library to traverse trees (a tree walker) and to attach information to tree nodes. The bulk of the implementation of EventB2Java is realised through the extension of the walker to generate Java code and JML specifications. Since Event-B includes mathematical types that are not built-in to Java or JML, EventB2Java implements them as Java classes in the form of a prelude library.

3.4.1 Tool installation

EventB2Java works with Rodin 3.5. To install EventB2Java, you should download the `EventB2JavaJml_plugin.jar` file available from the GitHub repository https://-github.com/ncatanoc/eventb2java and drop it into the `rodin.app/Contents/-Eclipse/dropins` local directory of Rodin.

3.4.2 Code generation

Code generation with EventB2Java works in the *Event-B* perspective of Rodin, but the generated code is available in the *Resource* perspective. To generate code for a specific Event-B machine using EventB2Java, in the Rodin Explorer panel of the *Event-B* perspective, the user right-clicks the machine to code-generate (see Figure 3.3), then selects *Java CodeGen*, and finally chooses either *Translate to multi-threaded Java* or *Translate to sequential Java* from the context menu. EventB2Java can generate code for machines at the different levels of abstraction, from the most abstract machine to the most concrete. It further uses ProB's constraint solver [3] to instantiate non-executable Event-B expressions with concrete Java values and produce a test file that runs the generated code, *e.g.*, the `Test_bin_m2.java` test class in Figure 3.4.
EventB2Java translates each event as a Java `Thread` class implementation. Events can be triggered when their guards are satisfied, which results in the execution of the event actions. EventB2Java models event triggering by creating two methods, a `guard_event` method containing the translation of the event guard, a `run_event` method containing the translation of the event body.

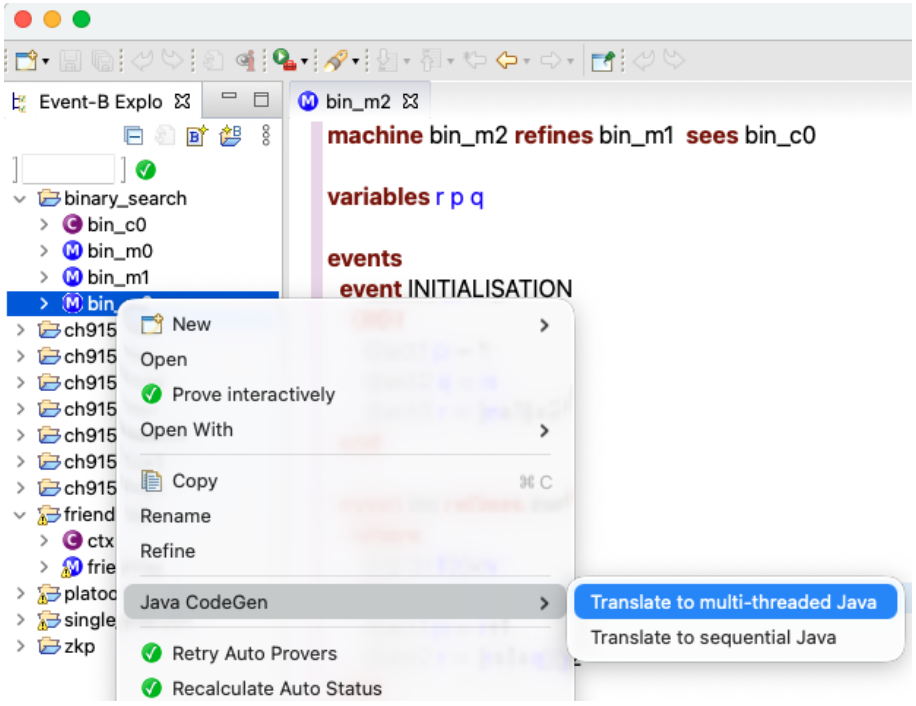

Fig. 3.3: EventB2Java: Java CodeGen

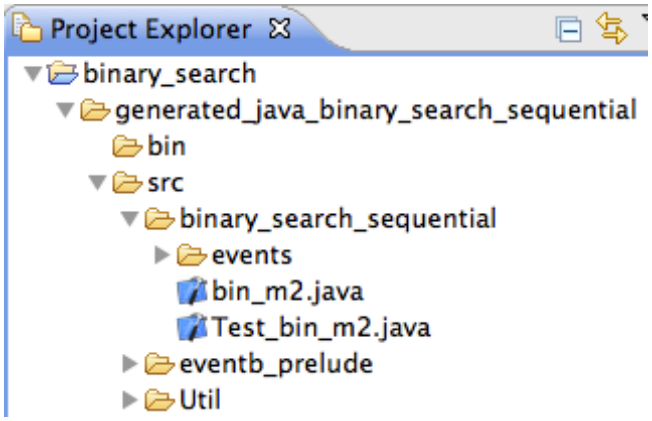

Fig. 3.4: EventB2Java: code structure

3.4.3 Eclipse support

The whole code generated by EventB2Java is an Eclipse project, which you can import from any Eclipse IDE for Java Developers https://www.eclipse.org/downloads/-

packages/ via `File` → `Import` → `General` → `Existing Projects into Workspace`. The Eclipse project includes the JML-specified Java code and the prelude libraries needed to execute the code. To access the code in Rodin, switch to the *Resource* perspective via `Window` → `Perspective` → `Open Perspective`. The Eclipse project includes a folder named `generated_java_binary_search` that contains the generated code, and an `eventb_prelude` sub-folder that contains the prelude Java implementation for Event-B sets, relations, booleans, etc. (see Figure 3.4). The `events` package of the `src` sub-folder contains the implementations of machine events. The file `bin_m2.java` (EventB2Java names this file after the corresponding machine in the Event-B model) contains the Java code that creates the objects of the classes that represent the events and that declares machine variables and constants. EventB2Java generates a test file `Test_bin_m2.java` that animates `bin_m2.java`. In `Test_bin_m2.java`, EventB2Java generates values for the constants so that they satisfy the axioms and theorems defined in the machine context, and generates values for the variables so that they satisfy the invariants of the machine.

3.5 Interactive proof in Rodin

Figure 3.1 shows the event upload of Poporo's abstract machine discussed in Example 2.1 (Page 10). The event is parameterised by a person p and a content item c. The bottom right part of the figure shows the *Symbols* toolbox for writing standard Event-B mathematical symbols. Rodin provides different *Perspectives*. The upper right part of Figure 3.1 shows that the current perspective is the *Event-B* perspective. You can either manually switch to a particular perspective (via `Window` → `Open Perspective`) or undertake an action that causes the editor to automatically switch to a default perspective for that action. In the left panel, discharged POs are depicted in green and undischarged POs in orange. If you double-click a PO on the left panel, then it opens in the *Proving* perspective of Rodin.

Rodin includes a list of predefined provers that assist users in discharging POs. A particular prover may or may not succeed in discharging a particular PO, and, therefore, the user might be forced to use a different prover, or to discharge parts of the PO manually.

Figure 3.2 shows Rodin's *Proving* perspective. The bottom part of the figure shows the various provers shipped with Rodin. The `PP` prover is a third-party prover shipped with AtelierB. `nPP` is a native prover to Rodin. A *lasso* adds more hypotheses to the current proof. Specifically, it adds hypotheses that share variables with the current goal. Provers can be used with different *forces*. `p0` (`PP` with force 0) feeds the `PP` prover with the selected hypotheses and with the current goal. `p1` (`PP` with force 1) performs a lasso operation before passing the hypotheses and goal to `PP`. `nPP` can be used with different forces too. The Rodin manual recommends using `nPP` before attempting any proof with `PP`. The `ML` prover is useful to discharge proofs involving arithmetic reasoning.

Next, we illustrate how to interactively discharge a proof obligation (PO) in Rodin. The example involves an invariant PO (see Rule INV in Page 19) for the event delete of the Poporo social network, shown in Figure 2.2.